

Smart Retry & Recovery for Subscriptions — Product Spec

A product spec for smarter recovery of failed subscription payments in India. Independent proposal based on public documentation; not affiliated with any company named here. Written from the point of view of a payment gateway serving many merchants.

1. The problem

Treating every payment failure the same way loses revenue that could have been recovered, or never lost at all.

A failed recurring payment does not mean a customer has churned. It may be temporary—insufficient funds or a bank outage—or require action, such as updating an expired card or re-authorizing a mandate.

Yet most platforms still handle every failure the same way:

| **Payment fails → wait → retry → repeat → stop.**

The problem is that **different failures need different recovery strategies**. A revoked mandate or expired card should not be retried blindly; it needs customer action. An insufficient-funds failure may succeed a few days later when funds arrive. A bank outage may recover within hours, making repeated retries during the outage wasteful.

The opportunity is not to retry more. It is to know when to retry, when to stop, and when to ask the customer to act, and to prevent the failures we can see coming.

How big it is, and why the failure reason matters

The pool is large and growing fast. UPI AutoPay alone processed 808 million mandate executions in July 2025, more than double the 392 million a year earlier.

Most failures are customer-side, not technical. NPCI data reported by Business Standard shows business declines averaging close to **74%** across the top 50

banks, for reasons such as insufficient funds rather than bank or network faults.

Recovery depends on the reason and the timing. Recurly studied more than 1,300 subscription businesses (card payments, mostly outside India, 2018) and found:

- Insufficient funds had the highest recovery rate of the top five decline reasons, and the three most common soft declines each recovered above 45%.
- Most recovery happened 2 to 12 days after the decline, and up to 15 days for temporary holds.
- The one hard decline in the top five, invalid card number, still recovered above 20%, but only through a card update, not a retry.

This matters because retry budgets are limited, and some Indian retry schedules concentrate attempts early in the recovery window. For example, Razorpay's card retry schedule attempts payment on T+1, T+2 and T+3 before halting the subscription. Cashfree runs UPI AutoPay retries at hourly intervals. Every attempt spent at the wrong time is one the merchant can't get back.

For one merchant with 50,000 subscribers paying ₹499 a month:

- Annual billing: **₹29.94 crore**
- Assuming **8–18%** of debits fail on the first attempt: **₹2.40–5.39 crore** a year enters the recovery funnel
- If recovery improves from an assumed **35% to 50%**, the extra recovered revenue is about **₹36–81 lakh a year**



Where these numbers come from. Assuming a 35% baseline recovery rate, which Phase 0 measures from 12 months of history, improving to 50%. The 15-point gain is not an aspiration: classes 1, 2 and 3 currently consume retry attempts that can never succeed, and on UPI AutoPay there are only four. Reallocating those attempts to the three classes that can actually recover is where the improvement comes from.

The exact percentage will vary by merchant, rail and customer base. The strategic point is more important: at subscription scale, even a single-digit failure rate can translate into a quite meaningful revenue recovery opportunity.

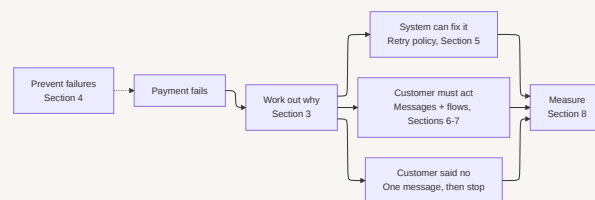
2. What we're building

In one sentence: Smart Retry & Recovery works out why each recurring payment failed, then chooses the right response: retry at the right time, ask the customer for the right fix, or stop.

Who it's for:

- **The merchant**, who wants lost revenue back without annoying customers, and without building separate logic for cards, UPI and eNACH.
- **The paying customer**, who wants their subscription to keep running, and to hear from us only when there's something useful to do.

How it works



Each box is a section of this doc:

1. **Work out why** (Section 3, Failure taxonomy). Every failure goes into one of seven recovery classes, based on the mandate's status and the rail's response code. This is the brain of the product.
2. **Stop failures before they happen** (Section 4, Prevention). Many failures can be seen coming: an expiring card, a price above the mandate limit, a billing date before payday.
3. **Fix what the system can fix** (Section 5, Retry policy). For insufficient funds, bank downtime and temporary limits, the recovery engine retries at the time most likely to work, within each rail's attempt limit. The recovery engine is the

part of the platform that runs after every failed debit and decides what happens next.

4. **Tell the customer the right thing** (Section 6, Dunning and communication). Each class gets a different message, at a different time, or none at all.
5. **Make the fix easy** (Section 7, Customer recovery flows). When the customer has to act: one page, one action, one authentication.
6. **Prove it works** (Section 8, Metrics). Recovery rate against a holdout, with a counter-metric for harm.

Two customers, start to finish

Priya and Rahul appear throughout this doc. Their classes are explained in Section 3.

	Priya: short of funds	Rahul: card expired
What happens	Her ₹499 OTT payment fails on the 1st. Her salary arrives on the 3rd	His ₹499 payment fails because his card expired, and the bank didn't move the mandate to his new card
Taxonomy	Class 4: insufficient funds	Class 1: dead instrument
Retry policy	Next attempt moved to the 3rd, not spent on the 1st	No retries: the card can't be charged
Messages	Nothing on the 1st beyond her bank's alert. On the 2nd: "We'll try again on the 3rd; please keep ₹499 available"	Straight away: "Your card has expired. Update it here." One reminder
Recovery flow	None needed. The retry on the 3rd succeeds	Three screens, one authentication, and a new mandate set up
Metrics	Counted in recovery rate and in retry success for class 4	Counted in recovery rate and in re-authorisation completion

Under today's fixed schedule, both get the same "payment failed, update your details" message. Priya's retries all land before her salary; Rahul's all hit a dead card.

What's in v1, and what isn't

In v1: the seven-class taxonomy; prevention checks before every debit; timed retries on UPI and cards within each rail's limits; class-specific messages; customer recovery flows including re-authorisation; and measurement against a holdout.

Not in v1 (see Section 9): automatic retries on eNACH, custom retry schedules for merchants, automatically charging a backup payment method, and machine-learned retry timing.

What already exists, and why this is different

Recovery isn't empty space. Every major Indian gateway retries failed recurring payments, and global billing platforms sell smart retries. The gap is narrower than "nobody does this": nobody publicly treats each rail's attempt cap as a budget allocated by cause, or re-decides when the cause changes mid-cycle.

What exists	How it works today (public docs)	What it doesn't do
Razorpay Subscriptions	Fixed card retry schedule, T+1, T+2, T+3, then the subscription halts	The same three attempts are spent whether the card is dead or the customer is paid on the 3rd
Cashfree Subscriptions	UPI AutoPay: 1+3 at hourly intervals. A newer "controlled" flow lets merchants send the pre-debit notification and trigger each attempt themselves	Hourly suits a bank outage, not insufficient funds. The controlled flow supplies the plumbing for custom retries, but the merchant has to build the logic
Dunning vendors	Messaging sequences and card-updater services on top of a gateway	They sit above the rail, so they cannot see mandate status, and mandate status is what decides whether a retry can ever work
Stripe Smart Retries	Messaging sequences and card updater services on top of a gateway	Does not model UPI AutoPay mandate state, attempt caps, or pre-debit notification rules

The difference in one line: existing products classify once and spend attempts on a schedule. This one re-decides at every attempt, and decides whether to spend an attempt at all, based on mandate state and response code rather than from a calendar.

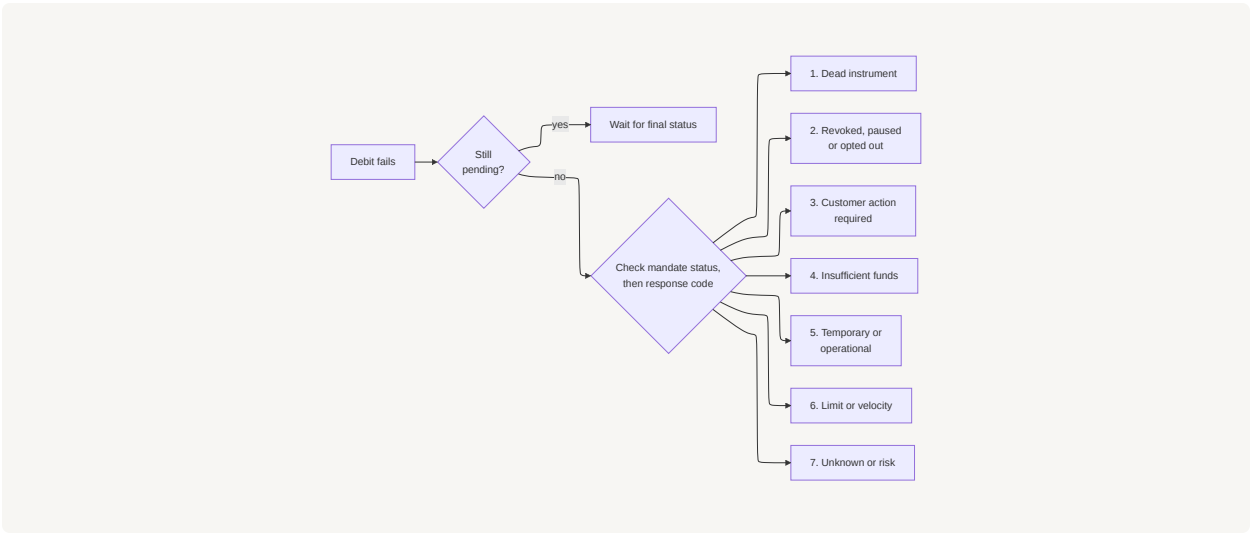
What they do better. All of them are integrated and running in production; this is a proposal. A merchant switching from a gateway default takes on integration work and risk for a gain that is modeled, not proven. That's why rollout starts in shadow mode with five design partners rather than with a pitch.

3. Failure taxonomy: why did it fail?

The problem section showed that the failure reason decides whether recovery is possible. This section defines those reasons.

Dozens of response codes and mandate statuses across three payment rails come down to **seven recovery classes**. The classes are based on **what we should do next**, not on the error code itself.

One rule sits above the taxonomy: a pending debit is not a failure. eNACH settles days later, and UPI and card debits can sit in a pending state. We wait for the final status before any retry or message, or we risk charging the customer twice.



Recovery class	How we detect it	Can an automatic retry succeed?	Correct recovery action	Recovery hypothesis*
1. Dead instrument	Cards: 14 (invalid account), 41 (lost), 43 (stolen), 54 (expired) where	No, not with the same instrument	For an expired card, first check whether the bank moved the mandate to the reissued card;	20–35%. Recurly found invalid-card declines recovered above 20%, only

Recovery class	How we detect it	Can an automatic retry succeed?	Correct recovery action	Recovery hypothesis*
	the mandate wasn't carried over to a reissued card. UPI: account closed or invalid. eNACH: 01 (account closed), 02 (no such account)		RBI allows this but doesn't require it. Otherwise stop and start a switch-payment flow: new card, UPI AutoPay or eNACH	through card updates. Offering UPI AutoPay should lift India above that; re-registration caps it
2. Revoked, paused or opted out	UPI: mandate revoked or paused. eNACH: mandate cancelled. Cards: cancelled-mandate error. All rails: the customer opted out of this debit from the pre-debit notification	Revoked or opted out: no. Paused: only after the resume date	Revoked: one neutral message, then stop. Paused: reschedule after the resume date. Opted out: one neutral message with a pay-now link for this payment	Revoked 5–15%: often deliberate. Opted out 10–20%: one payment, not the subscription. Paused 60–80%: the customer planned to return
3. Customer action required	Usually visible before the debit: invoice above the mandate maximum, mandate close to expiry, or amount needing AFA. At the debit:	No, not until the underlying state changes	Raise the limit, renew the mandate, approve the debit, or sort out the account with the bank. If the bank can't authenticate, send a pay-now link and offer	30–60%. Higher when the customer caused the change (an upgrade, a failed authentication); lower when their bank must act

Recovery class	How we detect it	Can an automatic retry succeed?	Correct recovery action	Recovery hypothesis*
	authentication failed, mandate expired, or account frozen, dormant or KYC-blocked		eNACH for future debits	
4. Insufficient funds	Cards: 51. UPI: insufficient balance. eNACH: return code 04	Yes , once money arrives	Don't retry back-to-back. Spread attempts across days, aimed at the customer's usual payment days; for credit cards, aim at the bill-payment date. Nudge before each attempt. Pay-now link if every attempt fails	40–55%. Recurly's card data puts insufficient-funds recovery above 45%. Four UPI attempts could push it lower; timing them to payday could push it higher
5. Temporary or operational	Bank or rail down: cards 91 (issuer unavailable), 96 (system error), timeouts; UPI technical declines, or NPCI rejecting an attempt during a blackout window. Our side: pre-debit notification missing, amount	Yes , once the issue is fixed	Outage: hold retries until the bank recovers, then retry within hours outside peak hours. Our error: fix the cause first, then restart the payment with a valid pre-debit notification	70–90%. The customer's money and intent haven't changed; only timing or our fix matters

Recovery class	How we detect it	Can an automatic retry succeed?	Correct recovery action	Recovery hypothesis*
	not matching the notification, missed bank cut-off, wrong mandate reference			
6. Limit or velocity	Cards: 61 (amount limit), 65 (frequency limit). Temporary limits only; a debit that permanently exceeds the mandate maximum is class 3	Yes, after the limit resets	Retry after the reset, usually the next day, at a different time of day. Offer pay-now if the customer wants to pay sooner	45–60%. Limits reset on a known schedule, so timing is easier than predicting payday
7. Unknown or risk	Cards: 05 (do not honour) without further detail, 59 (suspected fraud), issuer risk flags. Any unmapped code on any rail	Possibly, handled conservatively	One spaced retry at a different time, not a series, since repeated attempts can look like card testing. If it fails, ask the customer to check with their bank or pay another way. Review unmapped codes weekly	20–40%. Some are false positives that clear; some are real blocks. This class should shrink as mapping improves

** Recovery hypotheses are reasoned estimates, not measured Indian data. Phase 1 measures what each class recovers today; Phase 3 measures what the*

recommended action adds. Both replace these estimates (Section 10).

What the table shows

- **Only three classes are true retry problems (4, 5 and 6), and each needs a different timing:** days, hours or the next limit reset. A single schedule gets at least two of them wrong.
- **Classes 1, 2 and 3 need no automatic retry** until something changes. Retrying them wastes attempts that, on UPI, are capped at one plus three retries per mandate.
- **Most of class 3 is preventable.** Limit breaches, expiries and AFA thresholds are all visible before the debit. Section 4 covers how.

Priya lands in class 4, and **Rahul** in class 1.

4. Prevention: stopping failures before they happen

The taxonomy tells us why payments fail. Many of those reasons can be seen coming: a card about to expire, a price rise above the mandate limit, a billing date that lands before payday. The cheapest recovery is a failure that never happens.

Prevention happens at four moments in a subscription's life. Each moment catches different failures.

When	What we do	Stops
At sign-up	Let the customer choose a billing date near payday	4. Insufficient funds
	* Offer an optional backup payment method	1. Dead instrument; 5. Bank down
	Propose a mandate maximum that covers announced price changes	3. Over the limit
	Steer plans above ₹15,000 towards eNACH	3. Needs approval
	Use a mandate name the customer will recognise in their UPI app	2. Accidental cancellation
Before every debit	Check the amount against the mandate limit before the pre-debit notification goes out	3. Over the limit

When	What we do	Stops
	Check the amount matches the notification, and avoid bank cut-offs, UPI peak hours and maintenance windows	5. Our errors, planned downtime
	Use the same merchant name on every debit, and keep attempt patterns steady	7. Risk declines
Before a known deadline	Remind the customer before a card or mandate expires	1. Dead instrument; 3. Expired mandate
	Ask for approval in advance when a debit will need AFA	3. Needs approval
	Remind a week before a large annual renewal	2. Surprise opt-outs
After a failure, to stop the next one	Check whether a reissued card kept its mandate	1. Dead instrument
	Offer to move the billing date after two failed months in a row	4. Insufficient funds
	*Offer a backup method after a successful recovery	1. Dead instrument; 5. Bank down

The biggest lever: the billing date

Insufficient funds is likely the largest failure class, and most of it is a timing problem: the debit arrives before the salary does. The simplest fix is to let the customer choose when they're billed.

- **At sign-up:** "When would you like to be charged each month?", with the option to pick a date soon after payday.
- **After repeated failures:** if a customer fails two months running and pays a few days later each time, offer to move the date: "Move your payment to the 5th?"

Existing subscribers only get the second option; their billing date changes only if they choose.

The second lever: a backup payment method

A backup payment method—such as UPI AutoPay alongside a card—lets customers switch rails when their primary method fails.

When we offer it: at sign-up, as an optional step that's easy to skip; and straight after a successful recovery, when the customer has just felt the problem: "Add a backup so this doesn't happen again."

How it works in v1: when the main method fails for a reason a backup would solve (a dead card, a frozen account), we ask: "Your card couldn't be charged. Use your UPI AutoPay backup instead?" One tap approves it. The backup is already set up, so no new mandate is needed.

In v2: charging the backup automatically, for customers who explicitly agree to that at set-up. Refrain from using backup instrument for failures like insufficient funds.

For Priya, the billing-date offer is the real fix: after two months of paying on the 3rd, she's offered a move to the 4th, the day after her salary, and the failure stops happening. **For Rahul,** an expiry reminder would have let him update his card before anything failed.

5. Retry policy: how the system recovers what it can fix

Prevention won't stop every failure. For the classes the system can fix on its own (4, 5 and 6), the taxonomy says what each failure means; the retry policy decides what to do next. A fixed schedule treats retries as calendar events. This policy treats them as a limited budget to spend across the billing cycle, even when the reason for failure changes between attempts.

The decision loop

The recovery engine runs the same five steps after every failed attempt:

1. **Is the debit still pending?** If so, wait for the final status. No retry, no message.
2. **Classify it.** Check the mandate's status first, then the rail's response code. This sets the recovery class.
3. **Check the limits:** attempts left, permitted retry windows, pre-debit notification requirements, and whether the amount needs AFA (Appendix A).

4. **Choose the next best action.** If the class allows a retry, pick the permitted time with the best odds. If no time has a reasonable chance, **don't spend the attempt**; move the customer to the right recovery flow (Section 7).
5. **After the attempt, start again at step 1.** The new response may put the failure in a different class.

The key design choice is in step 4: **an attempt is only spent when it has a real chance of working**. A low-odds attempt isn't free. On UPI it uses up one of only four.

Budgets: one per debit, spent by class

Each rail gets an attempt limit per debit. Where regulation sets it, we follow it; where it doesn't, we set our own.

Rail	Attempts per debit	Why
UPI AutoPay	1 attempt + 3 retries	Set by NPCI, per mandate sequence, outside peak hours only
Cards	Our own limit, proposed 4	No RBI or NPCI cap. A limit protects customers from repeated failure alerts, and covers card-network retry rules where they apply
eNACH	No automatic retries in v1	Each failed debit can cost the customer ₹250–₹550 in bank charges, and results take days. eNACH failures go to messages and pay-now links instead (see Section 9)

Within that budget, the recovery class decides the spend:

Recovery class	Attempts it may use	Timing
4. Insufficient funds	All remaining	Spread over days, aimed at the days money usually arrives
5. Temporary or operational	1, rarely 2	Within hours of the bank recovering, or right after we fix our error
6. Limit or velocity	1	After the limit resets
7. Unknown or risk	1	Spaced out, then ask the customer
1, 2 and 3	0	Until the underlying problem is fixed

When to retry insufficient funds: within the times regulation allows, on the day the money is most likely to be there. That day comes from the customer's own past successful payments, or from similar customers if they're new. v1 uses this simple rule; a learned model is deliberately left for later (Section 9).

Priya's cycle: four decisions

Her ₹499 UPI AutoPay debit is due on the 1st:

When	What happened	Class	Decision	UPI attempts left
1st, 8 am	First attempt fails: insufficient balance	4. Insufficient funds	Don't retry today. Her past payments went through on the 3rd and 4th, so aim for the 3rd	3
2nd	Nudge sent: "keep ₹499 available for your subscription"	No change	Wait	3
3rd, 2 pm	Attempt hits a bank technical decline	5. Temporary	The reason changed. Wait for the bank to recover, then retry the same day outside peak hours	2
3rd, 10 pm	Attempt succeeds	Recovered	Stop	1 unused

A fixed schedule would have spent all four attempts on the 1st and lost the payment before her salary arrived.

Rahul gets no retries at all: class 1 goes straight to messages and the recovery flow.

6. Dunning and communication: what the customer hears

The retry policy handles what the system can fix. Everything else depends on the customer, so what we tell them matters as much as when we retry.

A customer with an expired card needs to act. A customer who was short of funds just needs time and a nudge. Treating both the same is how platforms turn involuntary churn into voluntary churn: we nag the customer who doesn't need to act, and fail to help the one who does. So every message is chosen by recovery class. The goal is fewer, more useful messages, not more of them.

Messaging principles

1. **Explain the reason and give one clear action.** Tell the customer what happened and what to do, if anything. Never tell them to change something that isn't the problem.
2. **Time the message around the recovery action.** Straight away when the customer must act, the day before a retry when they may need to add funds, and never during a bank outage.
3. **Silence means no push messages, not hiding the truth.** If the customer can't act, we send no SMS or email. But the merchant's app always shows the current payment status.
4. **Complement the bank's notices, don't duplicate them.** Banks already send pre-debit and transaction notifications. We add what they can't say: why it failed and what to do next.
5. **Keep messages branded and limited.** Messages come from the merchant the customer knows, capped at three per failed invoice.

Prevention messages (an expiring card, a mandate limit below the new price) follow the same principles; Section 4 covers what we prevent.

What each customer sees

Class	When and where	Example message	Follow-up
1. Dead instrument	Straight away, once we've confirmed the bank didn't move the mandate. SMS, email, in-app	"The card ending 4417 can't be charged any more. Update it to keep [Service] running: [link]"	Reminder after 3 days; a final one before the subscription stops
2. Revoked, paused or opted out	Once, straight away. Email, plus in-app status	Revoked: "Your AutoPay was cancelled. Didn't mean to? Turn it back on: [link]". Paused: "Your AutoPay resumes on the 10th." Opted out: "Pay here to keep [Service] active: [link]"	None
3. Customer action required	Before the debit for limits, expiry and approvals; straight after for a frozen account. SMS, email, in-app	"Your plan is now ₹799, above your AutoPay limit of ₹500. Update it so your next payment goes through."	One reminder before the deadline
4. Insufficient funds	Nothing on the day. SMS the day before the next attempt	"We'll try your ₹499 payment again on the 3rd. Please keep the amount available."	If every attempt fails: one pay-now link
5. Temporary or operational	In-app status only; a heads-up if the bank sends a second notice	"You may see a second notice from your bank. It's for the same ₹499, not an extra charge."	Pay-now link only if unpaid at the end of the cycle
6. Limit or velocity	In-app, straight away	"Your bank declined this ₹499 payment because of a card limit. We'll try again tomorrow."	If that fails: one SMS or email with a pay-now link
7. Unknown or risk	After the one spaced retry	"Your bank declined this payment as a security	One reminder

Class	When and where	Example message	Follow-up
	fails. Email, in-app	check. Please check with them, or pay another way: [link]". Never the word "fraud"	

While a debit is still pending, the app shows "payment processing" and nothing else goes out.

What the merchant sees

For every failed debit, the dashboard and webhook show the reason in plain words, the messages already sent, the next planned action and when, and the amount at risk. Merchants who run their own messaging can take the reason, recommended action and suggested wording through the API, and switch ours off.

Priya hears nothing on the 1st beyond her bank's alert, then one nudge on the 2nd. **Rahul** gets one clear message with a one-tap fix, which takes him to the recovery flow.

7. Customer recovery flows: what the customer does

Section 6 ends at the link. This section is about what happens after the customer taps it. The rule is simple: one reason, one page, one main action. And a recovery only counts when the unpaid amount is collected. A customer who fixes their card and then waits a month for the next debit is recovered on paper, not in revenue.

Design principles

1. **Land on the reason, not a form.** The page opens with why the payment failed, the amount due and one main button. "Pay another way" is always available as a second option.
2. **Fix the payment method and collect the invoice in one go.** RBI allows authentication to be combined when the first debit is processed along with a new mandate's registration, so one approval can cover both.
3. **Offer the rail most likely to work, not just the one that failed.** UPI AutoPay is quick to set up and suits amounts within the ₹15,000 AFA-free limit. eNACH

suits high-value plans, with debits up to ₹10 lakh, but can take 2–5 business days to activate.

4. **Never ask for what we already know.** Pre-fill the name, amount, plan and merchant. The customer only types what's genuinely new.

The flows

Flow	Offered when	What the customer does	What happens to the unpaid invoice
Update card*	Card expired, reissued without the mandate carried over, or blocked	Enters the new card details	Charged as the new mandate's first debit, with the same authentication
Switch to UPI AutoPay*	Dead instrument, frozen bank account, repeated bank problems, or the customer prefers it	Approves the mandate in their UPI app with their PIN	Charged as the first debit where the gateway supports it; otherwise straight after activation
Switch to eNACH*	High-value plans; amounts above the AFA-free limit	Authenticates through net banking or Aadhaar	Collected by pay-now in the same session, because eNACH activation can take days
Raise the mandate limit*	Invoice above the mandate's maximum after a price rise or upgrade	Approves a higher maximum	Paid now in the same session; a new debit would need a fresh pre-debit notification and a 24-hour wait
Reactivate AutoPay*	Mandate revoked by mistake, or expired	Sets up a new mandate	Charged as the new mandate's first debit
Approve this debit	Amount above the AFA-free limit	Authenticates before the deadline	Collected on the scheduled date once approved; pay-now if the deadline passes
Use my backup	The main method fails for a reason a backup solves, and the	Taps once to approve	Charged on the backup, after the usual pre-debit notification

Flow	Offered when	What the customer does	What happens to the unpaid invoice
	customer has one (Section 4)		
Pay now	Any class, as the fallback	Makes a one-time payment by any method	Paid, and every scheduled retry for that invoice is cancelled at once

**Needs mandate re-authorisation (see below).*

The hardest part: re-authorising a mandate

Five of these flows ask the customer to authenticate again for a subscription they already agreed to. RBI requires authentication to register or change any mandate, so the step can't be removed. The job is to make it feel like a quick confirmation, not a fresh sign-up, because every extra step is a chance to abandon.

- **Start where the customer already trusts the merchant:** inside the merchant's app, where they're logged in. For expiring mandates and limit changes, start before anything fails.
- **Make it feel like continuation, not sign-up.** Show the existing subscription prominently (plan, price, member since) and explain only what changed, in one plain line: "Your card ending 4417 has expired." We say "continue" and "confirm", never "subscribe".
- **Keep it short:** three screens and one authentication. The customer sees their subscription, confirms what's being authorised (same ₹499 a month, the maximum, and this month's payment), then approves in their own UPI app or bank.
- **Make sure it can't be mistaken for a scam.** "Authenticate to keep your subscription" is exactly what fraud messages say. Links go only to the merchant's own app or domain, PINs and OTPs are entered only in the customer's own UPI app or bank.
- **If the customer drops off:** save their progress, send one reminder, and keep "Pay this month" available so the subscription doesn't have to stop.

After the customer acts: Collect the invoice, stop all remaining retries and messages, restore the subscription, and tell the merchant straight away. Record which flow worked, so we learn over time.

Rahul taps the link, sees his subscription and the reason, switches to UPI AutoPay with one approval, and is offered a backup method on the final screen.

8. Metrics: how we'll know it worked

We judge the product on one headline metric, **recovery rate**, supported by six metrics and one counter-metric that catches customer harm. Every metric is reported for both the **new system and the holdout**, and we always look at the difference, because some failed payments recover without any help.

Definitions used throughout

- **Failed invoice:** the first debit attempt returned a final failure. Pending doesn't count until it's final. Each invoice counts once, however many attempts it gets.
- **Cycle:** from the invoice's due date to the next due date, in IST.
- **Recovered:** paid before its cycle ends, by any route (retry, pay-now, new mandate, backup). If refunded or charged back within 60 days, it no longer counts.

All metrics

Metric	Formula	What it tells us	Target	Split by
Recovery rate (North Star)	Recovered value ÷ failed value, per billing cycle	How much failed money we got back	+5 points vs holdout	Class, rail, merchant category
Involuntary churn rate	Subscriptions ended by the system with an unpaid failed invoice ÷ active subscriptions at month start	Whether we kept customers who wanted to stay	Below holdout	Rail, merchant category

Metric	Formula	What it tells us	Target	Split by
Prevented failures	(Holdout first-attempt failures – new-system first-attempt failures) per 1,000 scheduled debits	Whether we're stopping failures before they happen, not just recovering them	Above zero, especially classes 1, 3 and 4	Class, rail
Cost per recovered rupee	(Retry fees + message costs) ÷ recovered value, per ₹100	Whether recovery costs less than it brings back	Below holdout	Rail, channel
Days to recovery	Time paid – time of first failure, for recovered invoices	How long the customer is in limbo; faster recovery means fewer give up	Median below holdout	Class, recovery route
Retry success rate	Successful retries ÷ all retries, grouped by the class assigned before the attempt	Whether we retry the right failures at the right time	Tracked, no target	Class, rail, attempt number
Re-authorisation completion	Customers who finish re-authorising ÷ customers who start	Whether the hardest recovery flow works	Tracked, no target	Rail, flow
Dunning-attributed voluntary churn (counter-metric)	Customers who cancel or revoke within 7 days <i>after</i> our message ÷ customers who got at least one message	Whether our messages are pushing customers away	No higher than holdout	Class, channel

Stop rule for the counter-metric: we check it every week. If it's higher than the holdout for two weeks in a row, and the gap is statistically significant, we freeze the rollout at its current level and stop adding messages until we find and fix the cause.

How to read them: always against the holdout; always split by class, because a strong overall number can hide one class doing harm; and by value, not count, so a failed ₹18,000 renewal matters more than a failed ₹99 one. Events to log and edge cases are in Appendix B.



Testing it: the experiment

Hypothesis. Class-based recovery lifts recovery rate by at least 5 percentage points over the current fixed schedule, without increasing dunning-attributed voluntary churn.

Unit of randomization: the customer, not the invoice. A customer can hold several subscriptions with one merchant. Randomizing per invoice would put the same person in both arms, and our messages in one arm would change their behavior in the other.

Baseline. Assumed 35%. Phase 0 measures it from 12 months of history and Phase 1 measures it per class. The final design uses the measured number.

Minimum detectable effect: 5 points (35% to 40%). Section 1 sizes a 15-point gain; that is the full-program target, including prevention and dunning. Five points is the smallest gain from retry timing alone worth shipping. Below 3, the operational cost isn't justified.

Sample size. At 80% power and 5% significance: about 1,470 failed invoices per arm at a 50/50 split. At 90/10: about 820 in the holdout and 7,350 in treatment.

Duration. A merchant with 50,000 subscribers and an 8–18% failure rate produces 4,000–9,000 failed invoices a cycle. A 50/50 split reaches size in under one cycle; 90/10 takes one to two. Salary dates make any single cycle unrepresentative, so we run two cycles either way. We measure at 50/50 across the five design partners, then ramp to 90/10. The reason

for 50/50 is class-level power, not speed: a 10% holdout leaves the smaller classes too thin to read.

Reading it per class. The headline number will power first; class-level numbers will not. Insufficient funds is the largest class so it powers soonest, while limit or velocity and unknown or risk may never reach significance at a single merchant. For those, we pool across merchants and report them as directional rather than conclusive.

Guardrails. Dunning-attributed voluntary churn, with the stop rule above. Cost per recovered rupee must stay below the holdout.

Decision rule, committed before the test runs:

Result	Action
+5 points or more, guardrails flat	Ship, ramp to 90/10, move to the next segment
+3 to 5 points, guardrails flat	Ship the retry policy for classes 4–6 only; test prevention and messaging separately
Under +3 points, guardrails flat	Don't ship. Check classifier accuracy against Phase 1 data
Any gain, counter-metric up	Don't ship. The gain is coming from pressure on customers, not better timing

Risks. Seasonality, so at least two cycles. Novelty effects from new messages. Merchant mix: a result from low-ticket UPI merchants doesn't transfer to high-value plans, which is why those come in a later phase. Recovery rate is weighted by value, so for merchants with mixed ticket sizes we enlarge the sample accordingly.

Priya counts towards recovery rate and class 4 retry success; **Rahul** towards recovery rate and re-authorisation completion.

9. Trade-offs: what we're not building, and what we chose

v1 does three things well: stop retrying payments that can't succeed, retry the rest at the right time, and ask customers for the right fix. Everything below was left out

to protect that focus.

What we're not building in v1

Not in v1	Why	What v1 does instead
Automatic retries on eNACH	Results take days to arrive, there's no clear retry limit, and every failed debit can cost the customer a ₹250–₹550 bank charge. Most new volume is on UPI anyway	eNACH failures get the right message and a pay-now link. Timed retries cover UPI and cards only
Custom retry schedules for merchants	If merchants set their own retry days, the failure reason no longer decides the action, we can't measure impact cleanly, and a merchant could set up something non-compliant	One well-designed default. Merchants can turn it on or off, choose how long a subscription stays active after a failed payment, and switch our messages off
Automatically charging a backup payment method	It needs the customer's explicit consent for automatic use, and extra protection against charging twice	Customers who set up a backup switch to it with one tap when their main method fails (Section 4)
Machine-learned retry timing	We don't have the data to train it yet, and a simple rule likely gets most of the benefit	Retry on the day the customer's past payments usually go through

Also out of scope: WhatsApp messages, saving customers who choose to cancel, first-time mandate sign-up, and loan EMIs.

10. Rollout

We roll out in phases, start with the merchants who have the most to gain and the least to lose, and move existing merchants over without changing anything about their billing.

Who goes first

Low-ticket subscription merchants on UPI AutoPay, such as OTT, edtech and digital content, with monthly plans under ₹15,000:

- They have high volumes, so the holdout gives a clear answer quickly.

- Most of their failures are insufficient funds, where better timing helps most.
- Each failure is small, so any mistake costs little.

High-value plans, eNACH-heavy merchants and lenders come later, once v1 has proven itself.

Phase	When	What happens	We move on when
0. Audit and size	Weeks 1–4	Confirm what production does today on each rail. Size the opportunity from 12 months of history. Classify that history from stored response codes for a first, approximate baseline per class.	We have a real baseline per class, and a go or narrow-scope decision
1. Classify without acting	Weeks 5–6	Five design-partner merchants opt in. Their old retry schedule keeps running. The classifier labels each failure but doesn't act. We track which failures get paid, giving today's recovery rate per class. Tracking continues on the holdout through Phase 2, since a failure can take a month to resolve.	Recoverable failures are rarely labelled "never retry"
2. Stop doing harm	Weeks 7–12	No-retry rules for classes 1–3, fast retries for bank downtime, attempt limits per rail. Pre-debit checks on every debit: amount within the mandate limit, inside bank cut-offs, outside UPI peak hours and known bank maintenance windows, with a consistent merchant name	Zero retries on dead or revoked payments, and recovery no worse than the holdout; fewer operational failures than the holdout
3. Recover more	Weeks 13–26	Timed retries on UPI and cards, class-specific messages and recovery flows. Prevention messages: card and mandate expiry reminders, limit-breach prompts before the debit, reminders before large annual	Recovery rate up against the holdout, and dunning-attributed voluntary churn flat; fewer failures

Phase	When	What happens	We move on when
		renewals. Offers after a failure: move the billing date after repeated failures, and add a backup method after a recovery. Test: 50/50 against the holdout across the design partners for two cycles, then ramp to 90/10, keeping a 10% holdout (Section 8)	in classes 1 and 3 than the holdout
4. Expand	After week 26	On by default for new merchants; opt-in for existing ones. Other segments, starting with high-value plans. Sign-up changes, for merchants who adopt them: choice of billing date, optional backup method, a mandate name customers recognize, and steering high-value plans to eNACH	Each new segment passes the same checks as the first

Stop signals at any phase: any double charge, any retry on a dead or revoked payment, any debit without a valid pre-debit notification, or a sustained rise in dunning-attributed voluntary churn. The first three pause the rollout automatically.

Appendix

A. Rail rules reference

As of the RBI E-mandate Framework dated 21 April 2026 and NPCI OC-215/2025-26. These change often; check the latest circulars before external use.

Rule	Cards	UPI AutoPay	eNACH
Attempts per debit	No RBI or NPCI cap. Visa and Mastercard have retry rules; enforcement on Indian domestic e-mandates is unconfirmed	1 attempt + 3 retries per mandate sequence. Retries can fall on later days;	No cap found in primary sources

Rule	Cards	UPI AutoPay	eNACH
When debits can run	No restriction found	Outside peak hours only (peak: 10 am–1 pm and 5–9:30 pm)	Bank cut-offs apply
Pre-debit notification	At least 24 hours before each debit, stating amount and date and time	Same as cards	Outside the RBI e-mandate framework
Authentication-free limit	₹15,000 per debit; ₹1,00,000 for insurance, mutual funds and credit card bills	Same as cards	Not covered by the RBI framework
Setting up or changing a mandate	Needs authentication; the first debit can share it	Same as cards	Net banking or Aadhaar authentication
Reissued cards	Existing mandates <i>can</i> be moved to the new card; not required	n/a	n/a
Cost of a failed debit to the customer	None found	None found	Bank return charges of about ₹250–₹550

B. Metrics: events and edge cases

Every metric in Section 8 is built from six logged events:

Event	Key fields
<code>invoice_created</code>	invoice_id, subscription_id, amount, due_date, rail, experiment_arm
<code>attempt_made</code>	attempt_id, invoice_id, attempt_number, attempt_type, class_before_attempt, processing_fee
<code>attempt_result</code>	attempt_id, final_status, response_code, class_assigned, result_at
<code>message_sent</code>	message_id, invoice_id, customer_id, class, channel, delivery_cost, sent_at
<code>invoice_paid</code>	invoice_id, amount_paid, paid_at, route (retry, pay_now, new_mandate, backup, manual)
<code>subscription_ended</code>	subscription_id, ended_at, initiated_by (customer, merchant, system), reason

Edge case	How it's counted
Merchant collects the payment manually	Recovered, route = manual; reported separately
Plan changed mid-cycle and a new invoice issued	The old invoice leaves the total once the merchant cancels it
Customer cancels before recovery	Stays unrecovered; counted as voluntary churn, not involuntary
Recovered, then refunded or charged back within 60 days	Moved back to unrecovered
Still pending at the end of the cycle	Reported as "unresolved", neither failed nor recovered
Cancellation before our message (for example, the revocation that triggered it)	Not counted in the counter-metric: the cancellation must come after the message

C. Sources

- [RBI: Digital Payments – E-mandate Framework, 2026](#)
- [NPCI OC-215/2025-26, via MediaNama](#)
- [NPCI: AutoPay ecosystem statistics](#)
- [Razorpay: Payment retries](#)
- [Razorpay: Subscription states](#)
- [Cashfree: Charge a subscription](#)
- [Cashfree: Controlled subscription payments](#)
- [Stripe: India recurring payments](#)
- [Stripe: Invite-only in India](#)
- [Recurly: Top payment decline reasons \(2018\)](#)
- [Recurly: Churn rate benchmarks \(July 2026\)](#)
- [Recurly: UPI AutoPay](#)
- [Business Standard, citing NPCI](#)
- [PhonePe Business: NACH return charges](#)